

1) Prerrequisitos (tu máquina Arch Linux, Docker Desktop disponible)

Instalar / verificar:

- Docker (Docker Desktop ya lo usas): asegúrate de que está corriendo.
- minikube (recomendado driver: `docker`).
- kubectl.
- helm.
- terraform `>= 1.5`.
- git.
- python3 + pip (para la app demo).
- opcional: `hey` o `wrk` para carga (o k6).

Comandos ejemplo (Arch — ajusta según cómo instales paquetes):

```
Shell
# ejemplo de verificación
docker --version
minikube version
kubectl version --client
helm version
terraform -version
python3 --version
```

Si te hace falta instalar algo (ejemplos):

```
Shell
# pacman / paru / yay (según tu setup)
sudo pacman -Syu minikube kubectl helm terraform python-pip
# o con AUR helpers si aplica
```

2) Iniciar minikube

Usar driver docker (porque tienes Docker Desktop). Ajusta memoria/CPU según equipo.

Shell

```
minikube start --driver=docker --memory=6g --cpus=2
# chequea nodos
kubectl get nodes
```

Tenia un problema al hacer minikube start .. y me tocó instalar unos headers para el kernel, ya que no venia con el veth (que es importante para que Docker (y Kubernetes) puedan crear interfaces virtuales de red entre el host y los contenedores.) para que pudiera funcionar. Luego de reiniciar y comprobar el docker:

```
[diana@archlinux ~]$ lsmod | grep veth
[diana@archlinux ~]$ sudo modprobe veth
[diana@archlinux ~]$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

[diana@archlinux ~]$ minikube start --driver=docker --memory=6g --cpus=2
❗ minikube v1.36.0 on Arch
% Using the docker driver based on existing profile
❗ minikube 1.37.0 is available! Download it: https://github.com/kubernetes/minikube/releases/tag/v1.37.0
❗ To disable this notice, run: 'minikube config set WantUpdateNotification false'

❗ Starting "minikube" primary control-plane node in "minikube" cluster
❗ Pulling base image v0.0.47 ...
❗ docker "minikube" container is missing, will recreate.
❗ Creating docker container (CPUs=2, Memory=6144MB) ...
❗ Preparing Kubernetes v1.33.1 on Docker 28.1.1 ...
  ▪ Generating certificates and keys ...
  ▪ Booting up control plane ...
  ▪ Configuring RBAC rules ...
❗ Configuring bridge CNI (Container Networking Interface) ...
❗ Verifying Kubernetes components ...
  ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
❗ Enabled addons: storage-provisioner, default-storageclass
❗ Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
[diana@archlinux ~]$
```

Confirmamos lo siguiente:

- El contenedor se creó.
- Docker pudo establecer la red correctamente con **veth**.
- El demonio de Docker ya funciona de principio a fin.

Luego verifique nodos y ahora me aparece minikube

```
[diana@archlinux ~]$ kubectl get nodes
NAME       STATUS    ROLES    AGE   VERSION
minikube   Ready     control-plane  11m   v1.33.1
[diana@archlinux ~]$
```

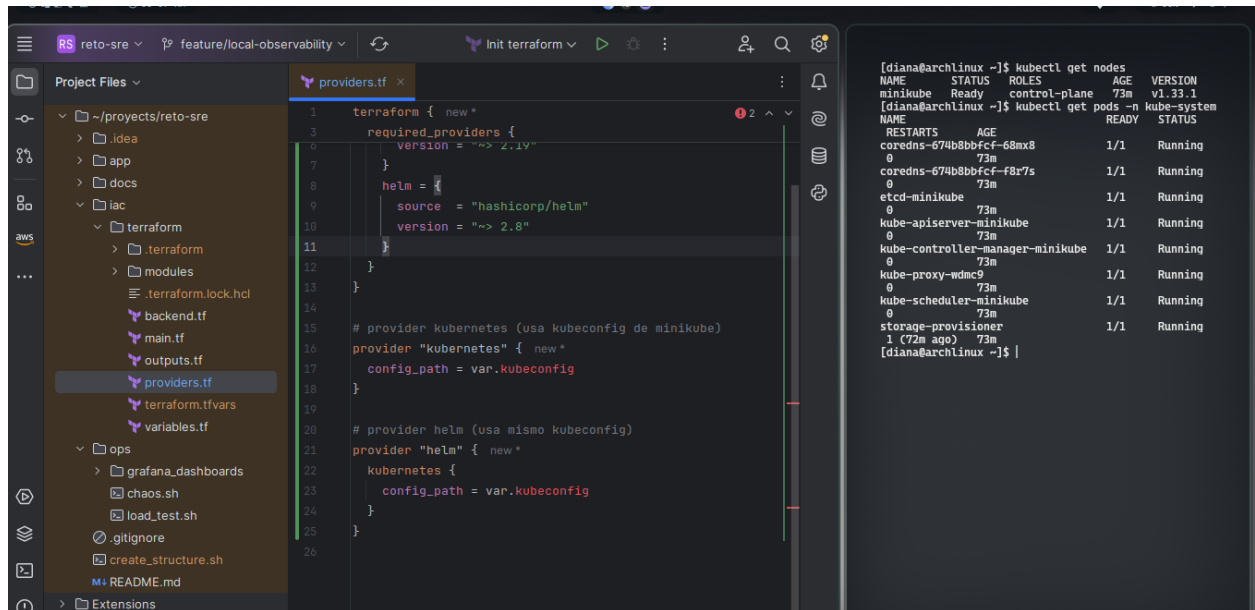
Luego habilitar ingress si lo usará:

```
[diana@archlinux ~]$ minikube addons enable ingress
❏ ingress is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
• Using image registry.k8s.io/ingress-nginx/controller:v1.12.2
• Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.5.3
• Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.5.3
❏ Verifying ingress addon ...
❏ The 'ingress' addon is enabled
[diana@archlinux ~]$
```

3) Estrategia mínima de cambios en tu repo (mantener buenas prácticas)

Git diff main - para ver que tengo de diferente desde mi rama actual a la main

Para mi caso, NO cree carpeta local, simplemente lo dejé igual y los providers estan en archivo 'providers.tf':



No rompas tu árbol principal. Añade una carpeta local bajo `iac/terraform/`:

```
None
/iac/terraform/
├── (cloud files...) main.tf providers.tf ...
└── local/
    ├── providers.tf
    ├── main.tf
    ├── variables.tf
    ├── outputs.tf
    └── helm-values/           # valores custom para kube-prometheus-stack,
otel
```

Razonamiento: así no tocas `backend.tf` ni S3/Dynamo/Terraform backend de cloud. Para despliegues locales trabajas desde `iac/terraform/local/` y para cloud desde `iac/terraform/` raíz.

4) Archivo `providers.tf` (`iac/terraform/local/providers.tf`)

Ejemplo mínimo (Terraform ≥ 1.5 , providers helm + kubernetes):

None

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    kubernetes = {
      source = "hashicorp/kubernetes"
      version = "~> 2.19"
    }
    helm = {
      source = "hashicorp/helm"
      version = "~> 2.8"
    }
  }
}

# provider kubernetes (usa kubeconfig de minikube)
provider "kubernetes" {
  config_path = var.kubeconfig
}

# provider helm (usa mismo kubeconfig)
provider "helm" {
  kubernetes {
    config_path = var.kubeconfig
  }
}
```

variables.tf (iac/terraform/local/variables.tf):

None

```
variable "kubeconfig" {
  type    = string
  default = "~/kube/config"
}

variable "namespace" {
  type    = string
  default = "payments"
}
```

Ahora, luego de esto, como se que estoy usando kubeconfig de minikube? en mi arch? O más bien, Terraform?

```
[diana@archlinux ~]$ echo $KUBECONFIG

[diana@archlinux ~]$ kubectl config get-contexts
CURRENT  NAME                                     AUTHINFO                                     CLUSTER                                     NAMESPACE
*        gke_desarrollo-3-454521_us-east4-a_gke-standard-cluster  gke_desarrollo-3-454521_us-east4-a_gke-standard-cluster  gke_desarrollo-3-454521_us-east4-a_gke-standard-cluster
minikube  minikube                                     minikube                                     default

[diana@archlinux ~]$
[diana@archlinux ~]$
[diana@archlinux ~]$ ~/.kube/config
bash: /home/diana/.kube/config: Permission denied
[diana@archlinux ~]$ sudo ~/.kube/config
[sudo] password for diana:
sudo: /home/diana/.kube/config: command not found
[diana@archlinux ~]$
[diana@archlinux ~]$ kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
minikube    Ready    control-plane  88m   v1.33.1
[diana@archlinux ~]$
```

1. Lo que tienes en `providers.tf`

```
provider "kubernetes" {
  config_path = var.kubeconfig
}
```

Esto significa que el provider de Terraform **no usa directamente `kubectl`**, sino que abre el archivo que le pases en `var.kubeconfig` (un `.yaml` con credenciales de acceso al cluster).

2. ¿Dónde está el kubeconfig de Minikube?

Por defecto, cuando corres `minikube start`, Minikube agrega su configuración al kubeconfig global:

Ubicación: `~/.kube/config`

Puedes confirmarlo ejecutando: `echo $KUBECONFIG`

- Si no devuelve nada, quiere decir que `kubectl` (y Terraform, si usas ese archivo) usa `~/.kube/config`.

Para ver si ahí está tu cluster de Minikube:

```
kubectl config get-contexts
```

- Verás una lista de contextos, y uno debería llamarse `minikube` y estar marcado como el *current*.

3. Cómo saber que Terraform usaría el kubeconfig de Minikube

Si en tu `terraform.tfvars` (o en variables de entorno) tienes algo como:

```
kubeconfig = "~/.kube/config"
```

- entonces efectivamente está leyendo el mismo archivo que usa `kubectl`.

Y como ya comprobaste con: `kubectl get nodes`

- que te devuelve el nodo `minikube`, significa que tu kubeconfig actual apunta al cluster de Minikube.

Por lo tanto, **Terraform también apuntará a Minikube** siempre y cuando `var.kubeconfig = ~/.kube/config`.

5) Instalar kube-prometheus-stack con Helm (Terraform helm_release)

Ejemplo `main.tf` (fragmento):

```
None
resource "helm_release" "kube_prometheus" {
  name          = "kube-prometheus-stack"
  repository    = "https://prometheus-community.github.io/helm-charts"
  chart         = "kube-prometheus-stack"
  namespace    = var.namespace
  create_namespace = true

  # valores recomendados en un archivo yaml
  values = [file("${path.module}/helm-values/kube-prom-values.yaml")]
}
```

En `helm-values/kube-prom-values.yaml` habilita:

- Grafana (admin/password por defecto que luego cambias).
- Prometheus operator (CRDs).
- Permitir ServiceMonitors (por defecto sí).

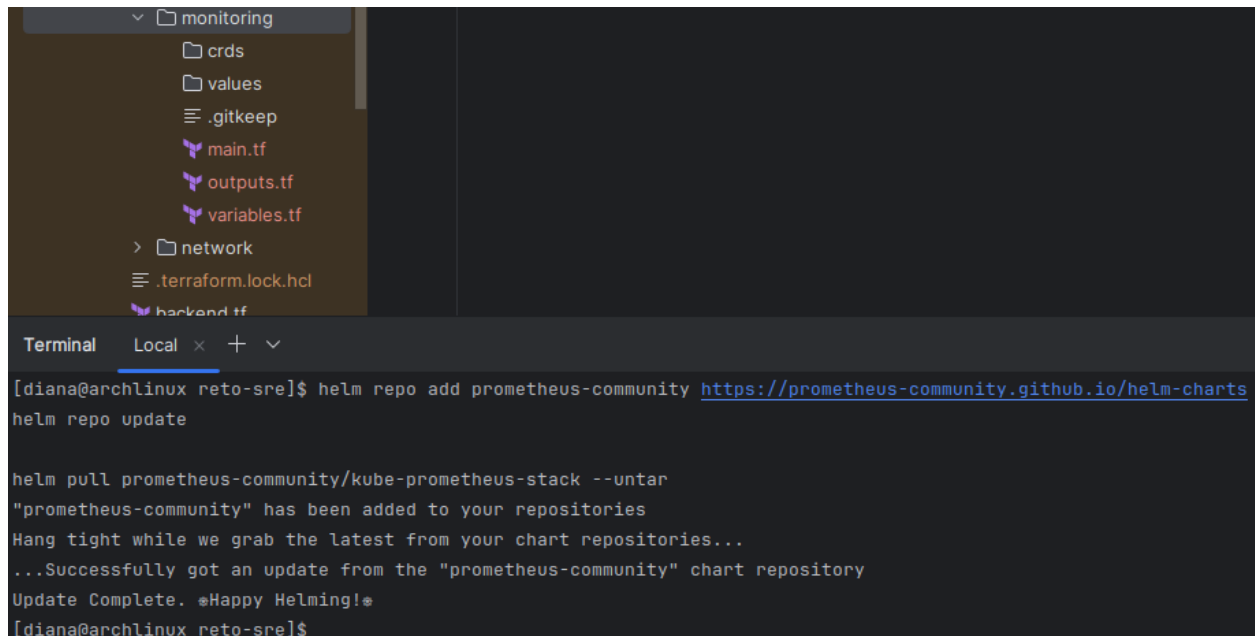
Ejemplo mínimo de valores (`helm-values/kube-prom-values.yaml`):

```
None
grafana:
  enabled: true
  adminUser: "admin"
```

```
adminPassword: "admin" # cambia en producción
prometheus:
  prometheusSpec:
    retention: "15d"
    scrapeInterval: "15s"
```

Nota: si prefieres instalar `kube-prometheus-stack` con `helm` CLI, también sirve; lo hacemos por Terraform para cumplir IaC.

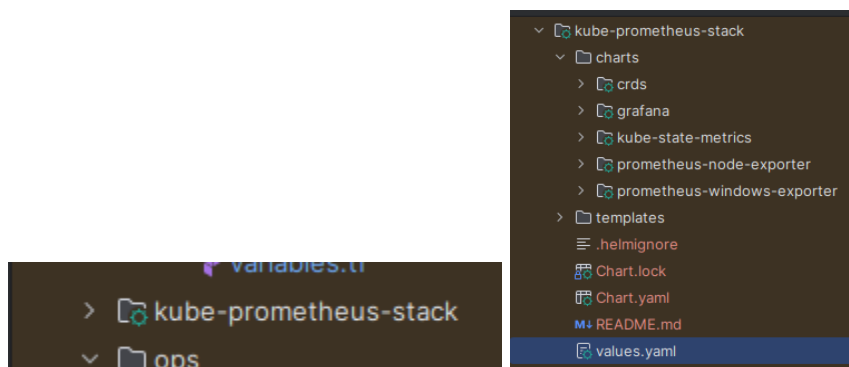
PAARA este paso, hice los siguientes subpasos ya que voy a crear la estructura con CRDs para mayor seguridad y estandar.



```
Terminal Local x + v
[diana@archlinux reto-sre]$ helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

helm pull prometheus-community/kube-prometheus-stack --untar
"prometheus-community" has been added to your repositories
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "prometheus-community" chart repository
Update Complete. ✨Happy Helming!✨
[diana@archlinux reto-sre]$
```

Esto va a crear una carpeta temporal llamada `kube-prometheus-stack/` justo en donde estás (raíz del repo).



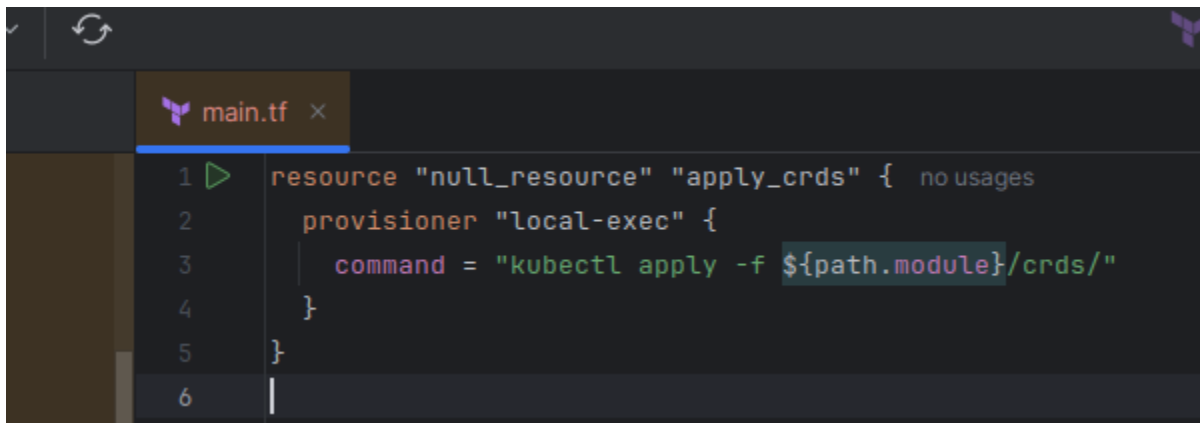
Y para no ensuciar mi repo, copie de esta carpeta temporal los crds y los pasé a mi carpeta de crds en monitoring

```
ondr-1.yaml crds files README.md templates values.yaml
[diana@archlinux crds]$ pwd
/home/diana/proyectos/reto-sre/iac/terraform/modules/monitoring/crds
[diana@archlinux crds]$ ls -lh
total 12K
-rw-r--r-- 1 diana diana 41 Sep 20 05:44 Chart.yaml
drwxr-xr-x 1 diana diana 444 Sep 20 05:44 crds
drwxr-xr-x 1 diana diana 16 Sep 20 05:44 files
-rw-r--r-- 1 diana diana 153 Sep 20 05:44 README.md
drwxr-xr-x 1 diana diana 38 Sep 20 05:44 templates
-rw-r--r-- 1 diana diana 118 Sep 20 05:44 values.yaml
[diana@archlinux crds]$
```

Luego borro el chart descargado: con `rm -rf kube-prometheus-stack`

Aplicar CRDs con Terraform antes del helm_release

Terraform no tiene recurso nativo `kubernetes_crd`, así que la forma segura es usar `kubernetes_manifest` o `kubectl apply` desde un `null_resource`.



```
1 resource "null_resource" "apply_crds" { no usages
2     provisioner "local-exec" {
3         command = "kubectl apply -f ${path.module}/crds/"
4     }
5 }
6
```

Creo el namespace y secret de grafana

```
main.tf x outputs.tf variables.tf
1 resource "null_resource" "apply_crds" {} no usages
2
3 }
4
5 /*-----Crear el namespace y secret para mi grafana-----*/
6
7
8 resource "kubernetes_namespace" "monitoring" { no usages
9   metadata {
10     name = var.namespace
11   }
12 }
13
14 resource "kubernetes_secret" "grafana_admin" { no usages
15   metadata {
16     name      = "grafana-admin"
17     namespace = var.namespace
18   }
19   data = {
20     admin-user      = base64encode(var.grafana_user)
21     admin-password = base64encode(var.grafana_password)
22   }
23 }
```

```
main.tf outputs.tf variables.tf x
1 variable "namespace" { no usages
2   default = "monitoring"
3 }
4
5 variable "grafana_user" { no usages
6   type    = string
7   sensitive = true
8 }
9
10 variable "grafana_password" { no usages
11   type    = string
12   sensitive = true
13 }
14
15 variable "chart_version" { no usages
16   type    = string
17   default = "65.5.0" # Pendiente mirar versión exacta
18 }
19
```

```
main.tf outputs.tf variables.tf x
1 variable "namespace" { no usages
2   default = "monitoring"
3 }
4
5 variable "grafana_user" { no usages
6   type    = string
7   sensitive = true
8 }
9
10 variable "grafana_password" { no usages
11   type    = string
12   sensitive = true
13 }
14
15 variable "chart_version" { no usages
16   type    = string
17   default = "65.5.0" # Pendiente mirar versión exacta
18 }
19
```

```
main.tf x outputs.tf variables.tf kube-prometheus-values.yaml
24 }
25
26
27 resource "helm_release" "kube_prometheus_stack" { no usages
28   depends_on = [null_resource.apply_crds] # los CDRs ya deben estar listos aquí en este punto
29
30   name      = "kube-prometheus-stack"
31   repository = "https://prometheus-community.github.io/helm-charts"
32   chart     = "kube-prometheus-stack"
33   version   = var.chart_version
34   namespace = var.namespace
35
36   values = [
37     file("${path.module}/values/kube-prometheus-values.yaml")
38   ]
39
40   wait      = true
41   timeout   = 600
42   atomic    = true
43
44   theus-stack
45 }
```

Luego exportamos el módulo de observabilidad en el main principal de terraform

```
monitoring/main.tf outputs.tf terraform/main.tf x
10 }
11
12 #Expo de modulos
13 module "monitoring" { new *
14   source = "../modules/monitoring"
15
16   namespace      = "monitoring"
17   grafana_user    = "admin"
18   grafana_password = "supersecure123"
19   chart_version   = "65.5.0"
20
21   providers = {
22     kubernetes = kubernetes.minikube
23     helm       = helm.minikube
24   }
25 }
26
```

Terraform aplica los **CRDs primero**.

Crea namespace y secrets.

Instala el chart con Helm, esperando a que todo quede listo.

Grafana usa tu secret sin exponer credenciales en el repo.

Paso fin para aplicar cambios: antes de aplicar terraform verifico que esten los cdrs en mi ruta:

Miro mis yaml..

```
[diana@archlinux reto-sre]$ ls iac/terraform/modules/monitoring/crds/
Chart.yaml  crds  files  README.md  templates  values.yaml
```

Pero me sale error: `kubectl apply -f iac/terraform/modules/monitoring/crds/`

```
[diana@archlinux reto-sre]$ kubectl apply -f iac/terraform/modules/monitoring/crds/
error validating "iac/terraform/modules/monitoring/crds/Chart.yaml": error validating data: kind not set; if you choose to ignore these errors, turn validation off with --validate=false
error validating "iac/terraform/modules/monitoring/crds/values.yaml": error validating data: [apiVersion not set, kind not set]; if you choose to ignore these errors, turn validation off with --validate=false
[diana@archlinux reto-sre]$
```

Esto anterior, no quedó así. Es decir, a la final me tocó eliminar la carpeta que cree de los CDRs y no hacerlo manualmente con `kubectl apply`, si no más bien que utilice los que viene, directamente. Por que si no, me salía un error en donde no sólo encontraba los yaml, si no también que no era como compatible.

Ahora, al aplicar Terraform init : me salen 2 cosas:

```
Terminal Local x
[diana@archlinux terraform]$ terraform init
Initializing the backend...
Initializing modules...

Warning: Reference to undefined provider

   on main.tf line 22, in module "monitoring":
   22:   kubernetes = kubernetes.minikube

There is no explicit declaration for local provider name "kubernetes" in module.monitoring, so Terraform is assuming you mean to pass a configuration for "hashicorp/kubernetes".

If you also control the child module, add a required_providers entry named "kubernetes" with the source address "hashicorp/kubernetes".

(and one more similar warning elsewhere)
```

Esto indica que no me reconoce el provider de k8s dentro de la ruta `/iac/terraform/modules/monitoring/providers.tf`. (pero si lo tengo el providers principal del terraform). Según las buenas prácticas hay que entender 2 conceptos:

1. Diferencia entre root y child modules

- **Root module** (`iac/terraform/` en tu caso)

Es el que realmente se ejecuta con `terraform plan/apply`. Aquí sí debes tener:

- `backend.tf` → define dónde se guarda el estado.
- `providers.tf` → define **cómo se configuran los providers concretos** (ejemplo: kubernetes con tu `~/ .kube/config`).
- `variables.tf`, `main.tf`, etc.

- **Child modules** (`iac/terraform/modules/monitoring/`)

Son módulos reutilizables. Un módulo nunca debería **configurar providers directamente**, sino **declarar cuáles requiere**.

Es decir, dentro de ellos pones `required_providers` pero **no los bloques provider**.

Ahora sí, me tocó hacer una configuración extras en cuanto al buen manejo de los child modules, añadir el k8s como provider en monitoring y cuadrar el secret para grafana

```
| Error: Backend configuration changed
|
| A change in the backend configuration has been detected, which may require migrating existing state.
|
| If you wish to attempt automatic migration of the state, use "terraform init -migrate-state".
| If you wish to store the current configuration with no changes to the state, use "terraform init -reconfigure".
|
[diana@archlinux terraform]$
```

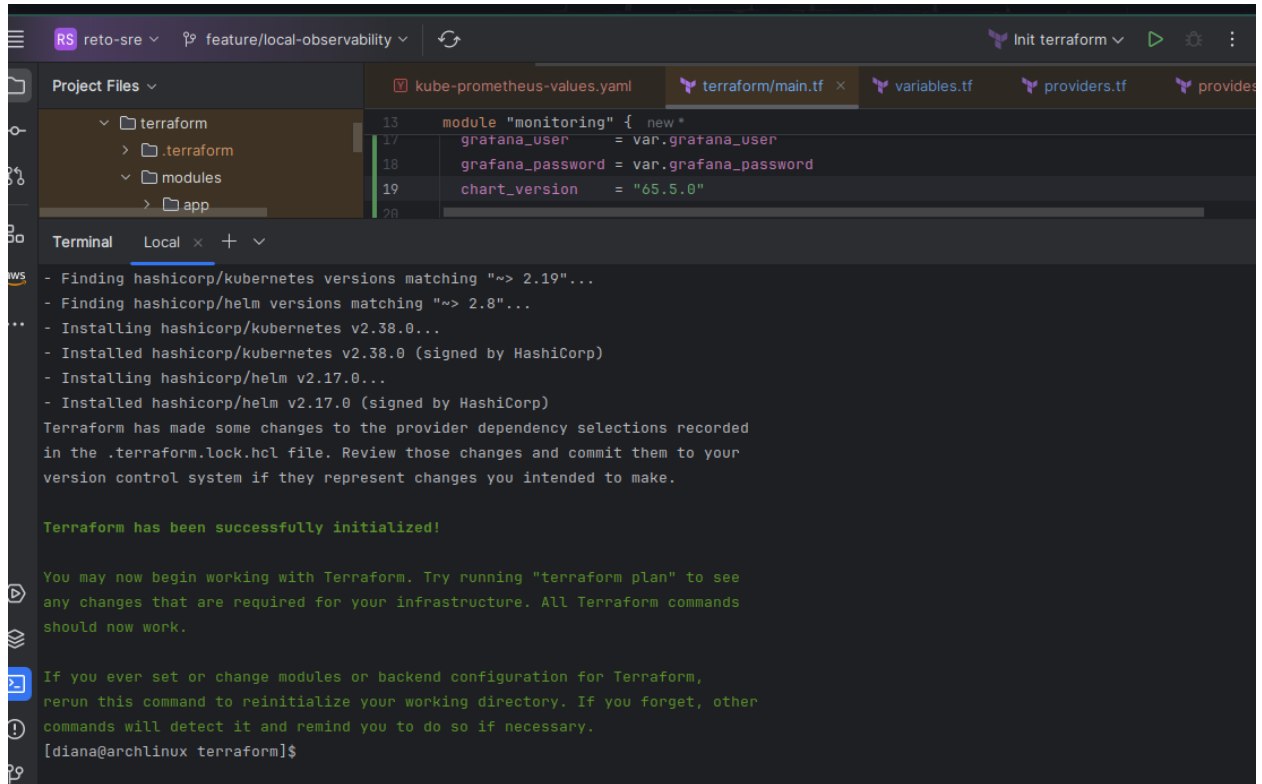
Esto pasa porque cambiaste la configuración de `backend.tf` (por ejemplo, si tenías S3 y ahora corres local con minikube). Tienes 2 opciones, según tu caso:

Si quieres trabajar con un backend diferente (por ejemplo, local para pruebas): `terraform init -reconfigure`

Si quieres mantener el backend actual y migrar el state al nuevo config: `terraform init -migrate-state`

En tu caso (como estás probando local con minikube), lo más seguro es usar: `terraform init -reconfigure`

Luego de esto, `terraform init`



The screenshot shows a code editor with a terminal window at the bottom. The terminal displays the output of the `terraform init` command. It lists the installation of `hashicorp/kubernetes` v2.38.0 and `hashicorp/helm` v2.17.0. It also shows a message about provider dependency selections in `.terraform.lock.hcl`. The terminal concludes with the message: **Terraform has been successfully initialized!** and provides instructions on how to use `terraform plan` and `terraform init` with flags.

```
13 module "monitoring" { new *
17   grafana_user = var.grafana_user
18   grafana_password = var.grafana_password
19   chart_version = "65.5.0"
20 }
```

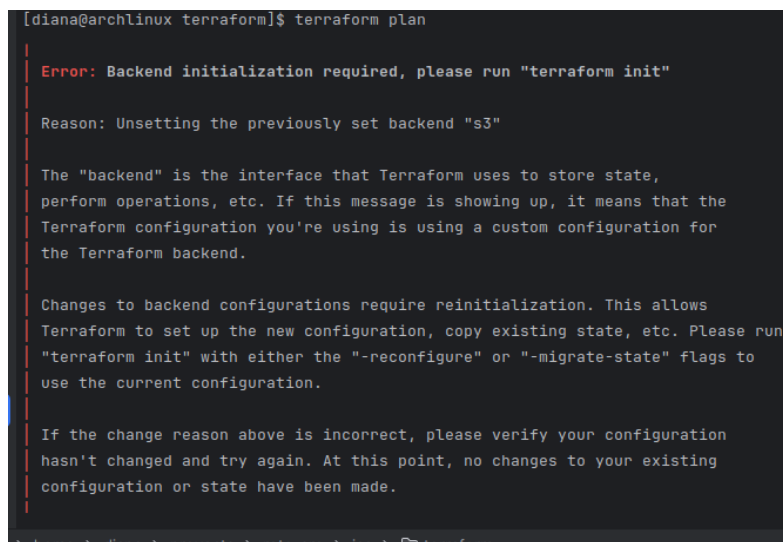
```
- Finding hashicorp/kubernetes versions matching "~> 2.19"...
- Finding hashicorp/helm versions matching "~> 2.8"...
- Installing hashicorp/kubernetes v2.38.0...
- Installed hashicorp/kubernetes v2.38.0 (signed by HashiCorp)
- Installing hashicorp/helm v2.17.0...
- Installed hashicorp/helm v2.17.0 (signed by HashiCorp)
Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
[diana@archlinux terraform]$
```

Pero luego:



The screenshot shows a terminal window with the command `terraform plan` executed. It results in an error: **Error: Backend initialization required, please run "terraform init"**. The reason given is "Unsetting the previously set backend "s3"". The terminal also provides a detailed explanation of the "backend" concept and instructions on how to resolve the issue by running `terraform init` with specific flags.

```
[diana@archlinux terraform]$ terraform plan

Error: Backend initialization required, please run "terraform init"

Reason: Unsetting the previously set backend "s3"

The "backend" is the interface that Terraform uses to store state,
perform operations, etc. If this message is showing up, it means that the
Terraform configuration you're using is using a custom configuration for
the Terraform backend.

Changes to backend configurations require reinitialization. This allows
Terraform to set up the new configuration, copy existing state, etc. Please run
"terraform init" with either the "-reconfigure" or "-migrate-state" flags to
use the current configuration.

If the change reason above is incorrect, please verify your configuration
hasn't changed and try again. At this point, no changes to your existing
configuration or state have been made.
```

Se da porque encontró restos de la configuración anterior de S3, entonces me tocó borrar el `.terraform/` y el `terraform.lock.hcl` (que se crea cada que se hace `init`)

Luego me toco hacer una pequeña modificación para que me tomara el secret de grafana. Y posteriormente

```
[diana@archlinux terraform]$ terraform plan

Error: Invalid function argument

on main.tf line 9, in resource "helm_release" "kube_prometheus":
  9:   values = [file("${path.module}/helm-values/kube-prom-values.yaml")]
     |         ^
     |         while calling file(path)
     |         path.module is "."

Invalid value for "path" parameter: no file exists at
"./helm-values/kube-prom-values.yaml"; this function works
only with files that are distributed as part of the
configuration source code, so if this file will be created by
a resource in this configuration you must instead obtain this
result from an attribute of that resource.

[diana@archlinux terraform]$
```

Esto se daba a que no encontraba el .yaml (que lo tenía declarado en el main del root), lo oquité de ahí y lo pase al módulo. Ahora:

```
[diana@archlinux terraform]$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource
actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

# helm_release.kube_prometheus will be created
+ resource "helm_release" "kube_prometheus" {
  + atomic                = false
  + chart                 = "kube-prometheus-stack"
  + cleanup_on_fail      = false
  + create_namespace     = true
  + dependency_update     = false
  + disable_crds_hooks   = false
  + disable_openapi_validation = false
  + disable_webhooks     = false
  + force_update         = false
  + id                   = (known after apply)
  + lint                 = false
  + manifest             = (known after apply)
  + max_history          = 0
  + metadata             = (known after apply)
}
```

```
Project Files
  terraform/main.tf
  monitoring/main.tf

Terminal
  Local
  +
  -

+ render_subchart_notes = true
+ replace                = false
+ repository             = "https://prometheus-community.github.io/helm-charts"
+ reset_values           = false
+ reuse_values           = false
+ skip_crds              = false
+ status                 = "deployed"
+ timeout                = 300
+ verify                 = false
+ version                = "77.10.0"
+ wait                   = true
+ wait_for_jobs          = false
}

# module.monitoring.helm_release.kube_prometheus_stack will be created
+ resource "helm_release" "kube_prometheus_stack" {
  + atomic                = true
  + chart                 = "kube-prometheus-stack"
  + cleanup_on_fail      = false
  + create_namespace     = false
  + dependency_update     = false
  + disable_crds_hooks   = false
  + disable_openapi_validation = false
  + disable_webhooks     = false
  + force_update         = false
  + id                   = (known after apply)
}
```

```
Project Files ▾ terraform/main.tf monitoring/main.tf ×
Terminal Local × + ▾
}

# module.monitoring.helm_release.kube_prometheus_stack will be created
+ resource "helm_release" "kube_prometheus_stack" {
+   atomic              = true
+   chart               = "kube-prometheus-stack"
+   cleanup_on_fail     = false
+   create_namespace    = false
+   dependency_update   = false
+   disable_crd_hooks   = false
+   disable_openapi_validation = false
+   disable_webhooks    = false
+   force_update        = false
+   id                  = (known after apply)
+   lint                = false
+   manifest            = (known after apply)
+   max_history          = 0
+   metadata            = (known after apply)
+   name                = "kube-prometheus-stack"
+   namespace           = "monitoring"
+   pass_credentials    = false
+   recreate_pods       = false
+   render_subchart_notes = true
+   replace             = false
+   repository          = "https://prometheus-community.github.io/helm-charts"
+   reset_values        = false
}
```

```
Project Files ▾ terraform/main.tf
Terminal Local × + ▾
+ skip_crds           = false
+ status              = "deployed"
+ timeout             = 600
+ values              = [
+   <<-EOT
+     grafana:
+       enabled: true
+       admin:
+         existingSecret: "grafana-admin"
+         userKey: "admin-user"
+         passwordKey: "admin-password"
+
+     prometheus:
+       prometheusSpec:
+         retention: "15d"
+         scrapeInterval: "15s"
+
+   EOT,
+ ]
+ verify              = false
+ version             = "65.5.0"
+ wait                = true
+ wait_for_jobs       = false

+ set {
+   name = "installCRDs"
+   value = "true"
}
```

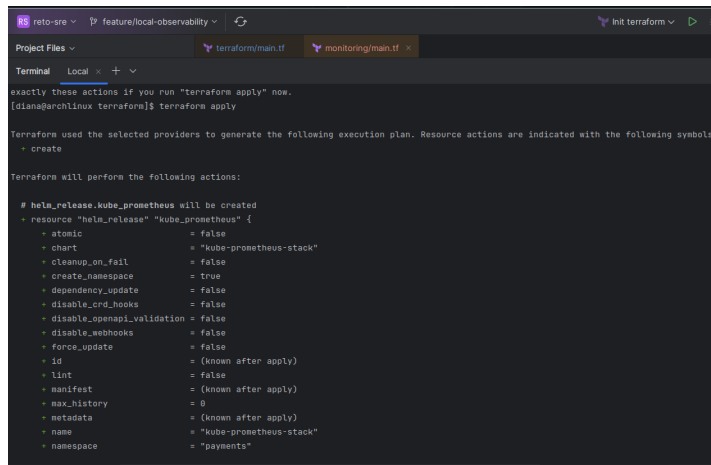
```
# module.monitoring.kubernetes_namespace.monitoring will be created
+ resource "kubernetes_namespace" "monitoring" {
+   id                  = (known after apply)
+   wait_for_default_service_account = false

+   metadata {
+     generation = (known after apply)
+     name       = "monitoring"
+     resource_version = (known after apply)
+     uid        = (known after apply)
+   }
+ }

# module.monitoring.kubernetes_secret.grafana_admin will be created
+ resource "kubernetes_secret" "grafana_admin" {
+   binary_data_wo      = (write-only attribute)
+   data                = (sensitive value)
+   data_wo             = (write-only attribute)
+   id                  = (known after apply)
+   type                = "Opaque"
+   wait_for_service_account_token = true

+   metadata {
+
+   }
}
```

Luego de aplica apply:



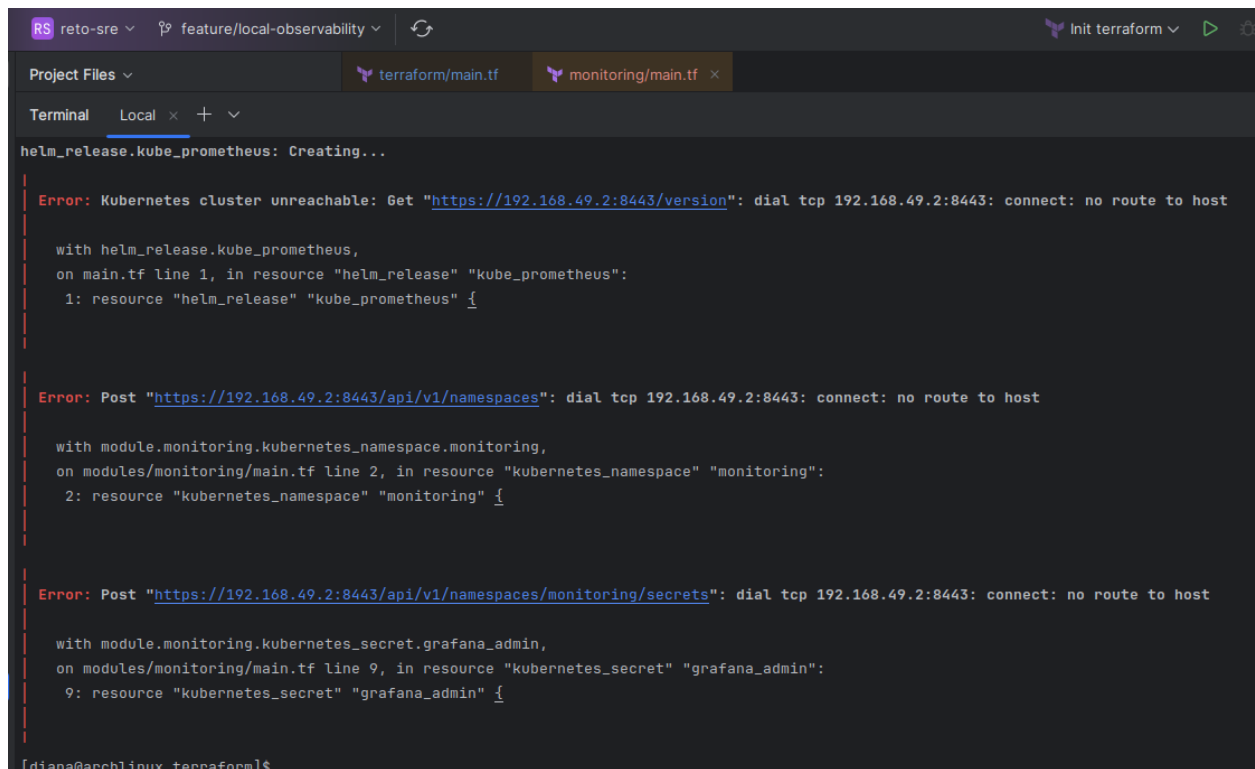
```
Project Files ▾ terraform/main.tf monitoring/main.tf x
Terminal Local x + ▾
[diana@archlinux terraform]$ terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# helm_release.kube_prometheus will be created
+ resource "helm_release" "kube_prometheus" {
  + atomic                = false
  + chart                 = "kube-prometheus-stack"
  + cleanup_on_fail      = false
  + create_namespace     = true
  + dependency_update    = false
  + disable_crd_hooks    = false
  + disable_openapi_validation = false
  + disable_webhooks     = false
  + force_update         = false
  + id                   = (known after apply)
  + lint                 = false
  + manifest             = (known after apply)
  + max_history          = 0
  + metadata             = (known after apply)
  + name                 = "kube-prometheus-stack"
  + namespace            = "payments"
}
```

Me sale lo siguiente : que parece ser de conexión al cluster



```
Project Files ▾ terraform/main.tf monitoring/main.tf x
Terminal Local x + ▾
[diana@archlinux terraform]$ terraform apply

helm_release.kube_prometheus: Creating...

Error: Kubernetes cluster unreachable: Get "https://192.168.49.2:8443/version": dial tcp 192.168.49.2:8443: connect: no route to host

with helm_release.kube_prometheus,
on main.tf line 1, in resource "helm_release" "kube_prometheus":
1: resource "helm_release" "kube_prometheus" {

Error: Post "https://192.168.49.2:8443/api/v1/namespaces": dial tcp 192.168.49.2:8443: connect: no route to host

with module.monitoring.kubernetes_namespace.monitoring,
on modules/monitoring/main.tf line 2, in resource "kubernetes_namespace" "monitoring":
2: resource "kubernetes_namespace" "monitoring" {

Error: Post "https://192.168.49.2:8443/api/v1/namespaces/monitoring/secrets": dial tcp 192.168.49.2:8443: connect: no route to host

with module.monitoring.kubernetes_secret.grafana_admin,
on modules/monitoring/main.tf line 9, in resource "kubernetes_secret" "grafana_admin":
9: resource "kubernetes_secret" "grafana_admin" {

[diana@archlinux terraform]$
```

significa que Terraform/Helm intenta hablar con tu cluster Kubernetes (la IP de Minikube en tu kubeconfig: 192.168.49.2), pero no puede conectarse y efectivamente:

```
[diana@archlinux ~]$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

[diana@archlinux ~]$ kubectl get nodes
E0920 10:28:36.026760 13110 memcache.go:265] "Unhandled Error" err="couldn't get curr
ent server API group list: Get \"https://192.168.49.2:8443/api?timeout=32s\": dial tcp
192.168.49.2:8443: connect: no route to host"
E0920 10:28:39.095662 13110 memcache.go:265] "Unhandled Error" err="couldn't get curr
ent server API group list: Get \"https://192.168.49.2:8443/api?timeout=32s\": dial tcp
192.168.49.2:8443: connect: no route to host"
E0920 10:28:42.168232 13110 memcache.go:265] "Unhandled Error" err="couldn't get curr
ent server API group list: Get \"https://192.168.49.2:8443/api?timeout=32s\": dial tcp
192.168.49.2:8443: connect: no route to host"
^Z
[1]+  Stopped                  kubectl get nodes
[diana@archlinux ~]$
```

Entonces inicié el minikube

```
[diana@archlinux ~]$ minikube status
minikube
type: Control Plane
host: Stopped
kubelet: Stopped
apiserver: Stopped
kubeconfig: Stopped

[diana@archlinux ~]$ minikube start
[[ ] minikube v1.36.0 on Arch
%- Using the docker driver based on existing profile host
[[ ] Starting "minikube" primary control-plane node in "minikube" cluster
[[ ] Pulling base image v0.0.47 ...
[[ ] Restarting existing docker container for "minikube" ...
[[ ] Preparing Kubernetes v1.33.1 on Docker 28.1.1 ...
[[ ] Verifying Kubernetes components ...
    * Using image gcr.io/k8s-minikube/storage-provisioner:v5
    * Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.5.3
    * Using image registry.k8s.io/ingress-nginx/controller:v1.12.2
    * Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.5.3
[[ ] Verifying ingress addon ...
[[ ] Enabled addons: storage-provisioner, ingress, default-storageclass
[[ ] Done! kubectl is now configured to use "minikube" cluster and "default" namespace b
y default
[diana@archlinux ~]$ minikube ip
192.168.49.2
[diana@archlinux ~]$ '[200-curl -k https://$(minikube ip):8443/version
bash: '$[200-curl': command not found
[diana@archlinux ~]$ curl -k https://$(minikube ip):8443/version
{
  "major": "1",
  "minor": "33",
  "emulationMajor": "1",
  "emulationMinor": "33",
  "minCompatibilityMajor": "1",
  "minCompatibilityMinor": "32",
  "gitVersion": "v1.33.1",
  "gitCommit": "8adc0f041b8e7ad1d30e29cc59c6ae7a15e19828",
  "gitTreeState": "clean",
  "buildDate": "2025-05-15T08:19:08Z",
  "goVersion": "go1.24.2",
  "compiler": "gc",
  "platform": "linux/amd64"
}
[diana@archlinux ~]$
```

luego de correr nuevamente el apply:

```
Project Files ▾ terraform/main.tf monitoring/main.tf x
Terminal Local x + ▾
Only 'yes' will be accepted to approve.

Enter a value: yes

module.monitoring.kubernetes_namespace.monitoring: Creating...
module.monitoring.kubernetes_secret.grafana_admin: Creating...
module.monitoring.kubernetes_namespace.monitoring: Creation complete after 0s [id=monitoring]
module.monitoring.kubernetes_secret.grafana_admin: Creation complete after 0s [id=monitoring/grafana-admin]
helm_release.kube_prometheus: Creating...
module.monitoring.helm_release.kube_prometheus_stack: Creating...
helm_release.kube_prometheus: Still creating... [00m10s elapsed]
module.monitoring.helm_release.kube_prometheus_stack: Still creating... [00m10s elapsed]
helm_release.kube_prometheus: Still creating... [00m20s elapsed]
module.monitoring.helm_release.kube_prometheus_stack: Still creating... [00m20s elapsed]
helm_release.kube_prometheus: Still creating... [00m30s elapsed]
module.monitoring.helm_release.kube_prometheus_stack: Still creating... [00m30s elapsed]
helm_release.kube_prometheus: Still creating... [00m40s elapsed]
module.monitoring.helm_release.kube_prometheus_stack: Still creating... [00m40s elapsed]
helm_release.kube_prometheus: Still creating... [00m50s elapsed]
module.monitoring.helm_release.kube_prometheus_stack: Still creating... [00m50s elapsed]
```

```
Project Files ▾ terraform/main.tf monitoring/main.tf x
Terminal Local x + ▾

Warning: Helm release "" was created but has a failed status. Use the 'helm' command to investigate the error, correct it, then run Terraform again.

with helm_release.kube_prometheus,
on main.tf line 1, in resource "helm_release" "kube_prometheus":
1: resource "helm_release" "kube_prometheus" {}

Error: services "kube-prometheus-stack-kube-proxy" not found

with helm_release.kube_prometheus,
on main.tf line 1, in resource "helm_release" "kube_prometheus":
1: resource "helm_release" "kube_prometheus" {}

Error: release kube-prometheus-stack failed, and has been uninstalled due to atomic being set: 15 errors occurred:
 * clusterroles.rbac.authorization.k8s.io "kube-prometheus-stack-grafana-clusterrole" already exists
 * clusterroles.rbac.authorization.k8s.io "kube-prometheus-stack-prometheus" already exists
 * clusterroles.rbac.authorization.k8s.io "kube-prometheus-stack-operator" already exists
 * clusterroles.rbac.authorization.k8s.io "kube-prometheus-stack-kube-state-metrics" already exists
 * clusterrolebindings.rbac.authorization.k8s.io "kube-prometheus-stack-operator" already exists
 * clusterrolebindings.rbac.authorization.k8s.io "kube-prometheus-stack-grafana-clusterrolebinding" already exists
 * clusterrolebindings.rbac.authorization.k8s.io "kube-prometheus-stack-kube-state-metrics" already exists
 * clusterrolebindings.rbac.authorization.k8s.io "kube-prometheus-stack-prometheus" already exists
 * services "kube-prometheus-stack-kube-controller-manager" already exists
```

El error que te está saliendo es muy común con **Helm + Terraform** cuando intentas hacer un **terraform apply** de nuevo y los recursos ya existen en el cluster.

👉 En tu caso, los logs dicen varias cosas importantes:

1. **services "kube-prometheus-stack-kube-proxy" not found**

Esto aparece porque Helm intenta administrar un servicio que no existe o que fue eliminado parcialmente.

2. **clusterroles.rbac.authorization.k8s.io "kube-prometheus-stack-grafana-clusterrole" already exists...**

services "kube-prometheus-stack-kube-etcd" already exists

Estos indican que ya hay recursos **creados manualmente o en un intento fallido anterior**, y cuando Helm trata de instalar de nuevo, **se encuentra con que ya existen**.

3. **release kube-prometheus-stack failed, and has been uninstalled due to atomic being set**

Esto significa que tienes **atomic = true** en tu **helm_release**, entonces si algo falla, Helm desinstala la release automáticamente.

Pero como algunos recursos quedaron colgados (ClusterRoles, Services, Webhooks), la siguiente instalación vuelve a chocar.

La opción que usé limpiar todo y reinstalar:

Pero esto aparentemente no sirve:

None

```
# Elimina la release de helm si existe
```

```
helm uninstall kube-prometheus-stack -n monitoring
```

```
# Limpia manualmente recursos que quedaron colgados
```

```
kubectl delete
```

```
clusterrole,clusterrolebinding,mutatingwebhookconfiguration,validatingwebhookconfiguration -l release=kube-prometheus-stack
```

```
# Limpia servicios
```

```
kubectl delete svc -n monitoring -l release=kube-prometheus-stack
```

```
[diana@archlinux reto-sre]$ kubectl config current-context
# Debe mostrar:
# gke_desarrollo-3-454521_us-east4-a_gke-standard-cluster
gke_desarrollo-3-454521_us-east4-a_gke-standard-cluster
[diana@archlinux reto-sre]$ kubectl config current-context
gke_desarrollo-3-454521_us-east4-a_gke-standard-cluster
[diana@archlinux reto-sre]$ helm uninstall kube-prometheus-stack -n monitoring
Error: Kubernetes cluster unreachable: Get "https://34.150.170.44/version": dial tcp 34.150.170.44:443: i/o timeout
[diana@archlinux reto-sre]$ kubectl config use-context minikube
Switched to context "minikube".
```

No hice nada practicamente y se soluciono el error anterior, pero ahora tengo

```
Terminal Local x + v
module.monitoring.helm_release.kube_prometheus_stack: Still creating... [03m01s elapsed]
module.monitoring.helm_release.kube_prometheus_stack: Still creating... [03m11s elapsed]
module.monitoring.helm_release.kube_prometheus_stack: Still creating... [03m21s elapsed]
module.monitoring.helm_release.kube_prometheus_stack: Still creating... [03m31s elapsed]
module.monitoring.helm_release.kube_prometheus_stack: Still creating... [03m41s elapsed]
module.monitoring.helm_release.kube_prometheus_stack: Still creating... [03m51s elapsed]
module.monitoring.helm_release.kube_prometheus_stack: Still creating... [04m01s elapsed]
module.monitoring.helm_release.kube_prometheus_stack: Still creating... [04m11s elapsed]
module.monitoring.helm_release.kube_prometheus_stack: Still creating... [04m21s elapsed]
module.monitoring.helm_release.kube_prometheus_stack: Creation complete after 4m27s [id=kube-prometheus-stack]

Error: Unable to continue with install: ClusterRole "kube-prometheus-stack-grafana-clusterrole" in namespace "" exists and cannot be imported into the current release: invalid ownership metadata; annotation validation error: key "meta.helm.sh/release-namespace" must equal "payments": current value is "monitoring"

with helm_release.kube_prometheus,
on main.tf line 1, in resource "helm_release" "kube_prometheus":
1: resource "helm_release" "kube_prometheus" {

[diana@archlinux terraform]$
acts > reto-sre > iac > terraform > modules > monitoring > main.tf > resource helm_release kube_prometheus_stack 22:14 LF UTF-8 AWS: default@us-west-2 2 space
```

Volví a hacer apply y me pc murió, pero:

```
[diana@archlinux ~]$  
[diana@archlinux ~]$ kubectl get svc -n monitoring  
NAME                                TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE  
alertmanager-operated              ClusterIP    None             <none>            9093/TCP,9094/TCP,9094/UDP 51m  
kube-prometheus-stack-alertmanager ClusterIP     10.103.251.250   <none>            9093/TCP,8080/TCP 51m  
kube-prometheus-stack-grafana      ClusterIP    10.100.239.210   <none>            80/TCP           51m  
kube-prometheus-stack-kube-state-metrics ClusterIP    10.104.94.83     <none>            8080/TCP         51m  
kube-prometheus-stack-operator      ClusterIP    10.103.96.133    <none>            443/TCP          51m  
kube-prometheus-stack-prometheus    ClusterIP    10.98.158.24     <none>            9090/TCP,8080/TCP 51m  
kube-prometheus-stack-prometheus-node-exporter ClusterIP    10.109.14.201    <none>            9100/TCP         51m  
prometheus-operated                ClusterIP    None             <none>            9090/TCP         51m  
[diana@archlinux ~]$  
[diana@archlinux ~]$  
[diana@archlinux ~]$ helm list -n monitoring  
NAME                NAMESPACE    REVISION    UPDATED           STATUS    CHART              kube-prometheus-stack-65.5.0    APP VERSION  
kube-prometheus-stack monitoring      1           2025-09-20 11:16:01.208672906 -0500 -05 failed kube-prometheus-stack-65.5.0    v0.77.2  
[diana@archlinux ~]$ kubectl get pods -n monitoring -w  
NAME                                READY    STATUS    RESTARTS   AGE  
alertmanager-kube-prometheus-stack-alertmanager-0 2/2      Running   2 (16m ago) 52m  
kube-prometheus-stack-grafana-585df99c6b-mz8gk    3/3      Running   4 (16m ago) 52m  
kube-prometheus-stack-kube-state-metrics-58bdf8474f-wbmqm 1/1      Running   3 (15m ago) 52m  
kube-prometheus-stack-operator-7c4fb6fc45-f27tk    1/1      Running   3 (15m ago) 52m  
kube-prometheus-stack-prometheus-node-exporter-lwch7 1/1      Running   2 (29m ago) 52m  
prometheus-kube-prometheus-stack-prometheus-0     2/2      Running   2 (16m ago) 52m
```

Add al release de helm la variable: update_force: true
Me tocó eliminar nuevamente la release porque ya se había creado y al aplicar el apply, salía el mismo error de la line 22.

```
module.monitoring.helm_release.kube_prometheus_stack: Creating...  
module.monitoring.helm_release.kube_prometheus_stack: Still creating... [ 00m10s elapsed]  
module.monitoring.helm_release.kube_prometheus_stack: Still creating... [ 00m20s elapsed]  
module.monitoring.helm_release.kube_prometheus_stack: Still creating... [ 00m30s elapsed]  
module.monitoring.helm_release.kube_prometheus_stack: Creation complete after 34s [id=kube-prometheus-stack]  
  
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.  
[diana@archlinux terraform]$
```

```
[diana@archlinux ~]$ kubectl get pods -n monitoring -w  
NAME                                READY    STATUS    RESTARTS   AGE  
alertmanager-kube-prometheus-stack-alertmanager-0 2/2      Running   0           50s  
kube-prometheus-stack-grafana-585df99c6b-7rsfm     3/3      Running   0           53s  
kube-prometheus-stack-kube-state-metrics-58bdf8474f-p62vb 1/1      Running   0           53s  
kube-prometheus-stack-operator-7c4fb6fc45-lb4dz    0/1      Pending   0           53s  
prometheus-kube-prometheus-stack-prometheus-0     2/2      Running   0           46s  
[diana@archlinux ~]$ kubectl get pods -n monitoring -w  
[diana@archlinux ~]$ kubectl get svc -n monitoring  
NAME                                TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE  
alertmanager-operated              ClusterIP    None             <none>            9093/TCP,9094/TCP,9094/UDP 67s  
kube-prometheus-stack-alertmanager ClusterIP     10.98.189.24     <none>            9093/TCP,8080/TCP 70s  
kube-prometheus-stack-grafana      ClusterIP    10.98.153.22     <none>            80/TCP           70s  
kube-prometheus-stack-kube-state-metrics ClusterIP    10.110.109.225   <none>            8080/TCP         70s  
kube-prometheus-stack-operator      ClusterIP    10.101.92.39     <none>            443/TCP          70s  
kube-prometheus-stack-prometheus    ClusterIP    10.97.80.203     <none>            9090/TCP,8080/TCP 70s  
kube-prometheus-stack-prometheus-node-exporter ClusterIP    10.101.192.111   <none>            9100/TCP         70s  
prometheus-operated                ClusterIP    None             <none>            9090/TCP         64s  
[diana@archlinux ~]$ helm list -n monitoring  
NAME                NAMESPACE    REVISION    UPDATED           STATUS    CHART              kube-prometheus-stack-65.5.0    APP VERSION  
kube-prometheus-stack monitoring      1           2025-09-20 12:25:52.390895634 -0500 -05 deployed kube-prometheus-stack-65.5.0    v0.77.2  
[diana@archlinux ~]$
```

Posterior, verifico los pods y servicios pero como se ve a continuación, tengo el pod de prometheus pendiente:

EL primer comando es para ver el nombre de los pods

```
[diana@archlinux ~]$ kubectl get pods -n monitoring
NAME                                READY   STATUS    RESTARTS   AGE
alertmanager-kube-prometheus-stack-alertmanager-0  2/2     Running   0           4m41s
kube-prometheus-stack-grafana-585df99c6b-7rsfm      3/3     Running   0           4m44s
kube-prometheus-stack-kube-state-metrics-58bdf8474f-p62vb  1/1     Running   0           4m44s
kube-prometheus-stack-operator-7c4fb6fc45-l8ktw     1/1     Running   0           4m44s
kube-prometheus-stack-prometheus-node-exporter-lb4dz  0/1     Pending   0           4m44s
prometheus-kube-prometheus-stack-prometheus-0      2/2     Running   0           4m37s
[diana@archlinux ~]$ kubectl get svc -n monitoring
NAME                                TYPE               CLUSTER-IP      EXTERNAL-IP      PORT(S)                                AGE
alertmanager-operated              ClusterIP          None             <none>            9093/TCP, 9094/TCP, 9094/UDP          4m56s
kube-prometheus-stack-alertmanager ClusterIP          10.98.189.24    <none>            9093/TCP, 8080/TCP                  4m59s
kube-prometheus-stack-grafana      ClusterIP          10.98.153.22    <none>            80/TCP                              4m59s
kube-prometheus-stack-kube-state-metrics ClusterIP          10.110.109.225  <none>            8080/TCP                            4m59s
kube-prometheus-stack-operator     ClusterIP          10.101.92.39    <none>            443/TCP                             4m59s
kube-prometheus-stack-prometheus   ClusterIP          10.97.80.203    <none>            9090/TCP, 8080/TCP                  4m59s
kube-prometheus-stack-prometheus-node-exporter ClusterIP          10.101.192.111  <none>            9100/TCP                            4m59s
prometheus-operated                ClusterIP          None             <none>            9090/TCP                             4m53s
```

Este pod corresponde a **Prometheus Node Exporter**, que es el componente que **recoge métricas del nodo donde corre** (CPU, memoria, disco, red, etc.).

El hecho de que esté en **Pending** significa **que Kubernetes no pudo programar el pod en ningún nodo**. Esto **no tiene que ver con tu Grafana o Prometheus principal**, pero sí significa que no se están recolectando métricas del nodo todavía.

Luego del diagnostico con los comandos:

```
kubectl describe pod kube-prometheus-stack-prometheus-node-exporter-lb4dz -n monitoring
```

```
kubectl get nodes -- aquí me sale el nodo de minikube solamente
```

```
kubectl describe node minikube
```

El mensaje clave en los eventos del pod pendiente es este:

```
0/1 nodes are available: 1 node(s) didn't have free ports for the requested pod ports.
```

Qué significa

- El **Node Exporter** quiere usar **el puerto 9100 en el host (HostPort: 9100)** en **todos los nodos**.
- Minikube ya tiene algo usando ese puerto (probablemente otro Node Exporter, o algún proceso local).
- Kubernetes no puede programar el pod porque **no puede asignarle el puerto 9100 en ese nodo**. Por eso el pod queda en **Pending**.

Esto **no es un problema de recursos** (CPU/memoria) ni de nodos, sino un conflicto de puerto en el nodo.

LO QUE HICE, fue añadir al values.yaml en monitoring el expose del puerto para que no haya conflictos de eso.
prometheus-node-exporter:

```
hostPort: 9101
```

Y antes de hacer apply nuevamente: `helm uninstall kube-prometheus-stack -n monitoring`.

SIN EMBARGO, ESTO ANTERIOR NO FUNCIONÓ, PERO NO ES BLOQUEANTE:

1 Qué métricas sí funcionan

- **Cualquier pod dentro del cluster** que exponga métricas a Prometheus (por ejemplo, tu demo en Python con OpenTelemetry o cualquier microservicio) **sí se van a recolectar**.
- Prometheus no depende de Node Exporter para estas métricas; solo hace scraping de los endpoints que definas en tus **ServiceMonitors** o **PodMonitors**.
- Los dashboards de Grafana que muestren estas métricas **funcionarán normalmente**.

2 Qué métricas no tendrás

- Métricas que provienen directamente del nodo (CPU, memoria, filesystem, disco, etc.) **no estarán disponibles** mientras Node Exporter esté Pending.
- Dashboards de Grafana que dependan de métricas de nodo aparecerán vacíos o incompletos.

3 Impacto en tu workflow actual

- Tu demo de Python y cualquier microservicio instrumentado con OpenTelemetry **sí va a reportar métricas a Prometheus y se verán en Grafana**.
- Solo perderías la visibilidad del "estado físico del nodo" (como si quieres ver CPU total, memoria libre del nodo, etc.).

VERIFICACIÓN DE PROMETHEUS:

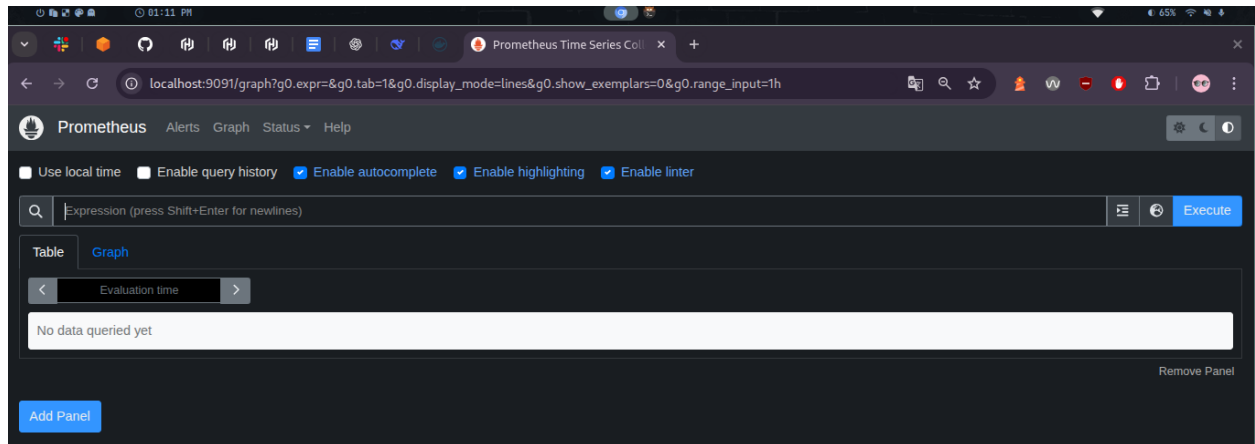
Comando: `kubectl port-forward svc/kube-prometheus-stack-prometheus 9091:9090 -n monitoring`

Porque Viendo el `kubectl get svc -n monitoring`, los servicios de **Grafana** y **Prometheus** están listos y con **ClusterIP**.

Como ya tenías problemas con el puerto 9090 ocupado, podemos hacer port-forward usando **puertos locales diferentes** para no chocar:

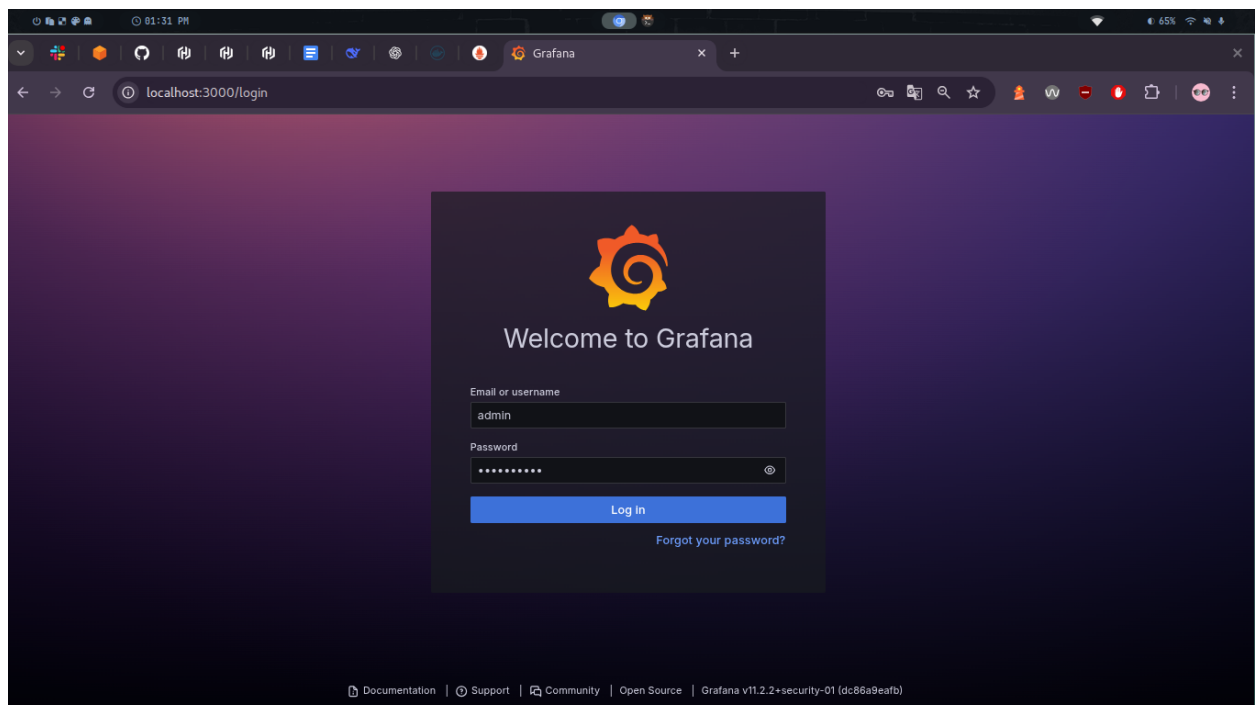
```
[diana@archlinux ~]$ kubectl port-forward svc/kube-prometheus-stack-prometheus 9090:9090 -n monitoring
Unable to listen on port 9090: Listeners failed to create with the following errors: [unable to create listener: Error listen tcp4 127.0.0.1:9090: bind: address already in use unable to
create listener: Error listen tcp6 [::]:9090: bind: address already in use]
error: unable to listen on any of the requested ports: [{9090 9090}]
[diana@archlinux ~]$ kubectl port-forward svc/kube-prometheus-stack-prometheus 9091:9090 -n monitoring
Forwarding from 127.0.0.1:9091 -> 9090
Forwarding from [::]:9091 -> 9090
Handling connection for 9091
Handling connection for 9091
Handling connection for 9091
```

Mantén estas terminales abiertas mientras uses los dashboards. Si las detienes, el port-forward también se detiene.



Para evitar conflictos de puertos: `ss -tuln | grep 3000` verificar siempre el proceso que los está ocupando
`kill -9 ID_PROCESO` y volver a correr `[diana@archlinux ~]$ kubectl port-forward svc/kube-prometheus-stack-grafana 3000:80 -n monitoring`

```
[diana@archlinux ~]$ ss -tuln | grep 3000
tcp    LISTEN 0      4096      127.0.0.1:3000      0.0.0.0:*    users:((("kubectl",pid=46221,fd=7))
tcp    LISTEN 0      4096      [::]:3000          [::]:*       users:((("kubectl",pid=46221,fd=8))
[diana@archlinux ~]$ kill -9 46221
[diana@archlinux ~]$ kubectl port-forward svc/kube-prometheus-stack-grafana 3000:80 -n monitoring
Forwarding from 127.0.0.1:3000 -> 3000
Forwarding from [::]:3000 -> 3000
Handling connection for 3000
Handling connection for 3000
```



YA que no me reconoce el user ni pass:

Validando variables de entorno

```
[diana@archlinux ~]$ kubectl get pods -n monitoring
NAME                                READY   STATUS    RESTARTS   AGE
alertmanager-kube-prometheus-stack-alertmanager-0 2/2     Running   0          51m
kube-prometheus-stack-grafana-585df99c6b-cpk4j    3/3     Running   0          51m
kube-prometheus-stack-kube-state-metrics-58bdf8474f-glqq9 1/1     Running   0          51m
kube-prometheus-stack-operator-7c4fb6fc45-vxhls   1/1     Running   0          51m
kube-prometheus-stack-prometheus-node-exporter-gwqfz 0/1     Pending   0          51m
prometheus-kube-prometheus-stack-prometheus-0    2/2     Running   0          51m
[diana@archlinux ~]$ kubectl describe pod kube-prometheus-stack-grafana-585df99c6b-cpk4j -n monitoring
Name:         kube-prometheus-stack-grafana-585df99c6b-cpk4j
Namespace:    monitoring
...
Environment:
  POD_IP:              (v1:status.podIP)
  GF_SECURITY_ADMIN_USER: <set to the key 'admin-user' in secret 'grafana-admin'> Optional: false
  GF_SECURITY_ADMIN_PASSWORD: <set to the key 'admin-password' in secret 'grafana-admin'> Optional: false
  GF_PATHS_DATA:        (/var/lib/grafana/)
```

6) Instalar OpenTelemetry Collector (receives OTLP, exporta a Jaeger/Grafana/traces)

Recomendado instalar una instancia simple del collector (helm chart `open-telemetry/opentelemetry-collector`) o usar `jaeger` helm. Ejemplo en Terraform:

```
None
resource "helm_release" "otel_collector" {
  name      = "otel-collector"
  repository = "https://open-telemetry.github.io/opentelemetry-helm-charts"
  chart     = "opentelemetry-collector"
  namespace = var.namespace

  values = [file("${path.module}/helm-values/otel-values.yaml")]
}
```

`otel-values.yaml` ejemplo (OTLP receiver y log export simple):

```
None
service:
  pipelines:
```

```
traces:
  receivers: [otlp]
  processors: [batch]
  exporters: [jaeger]
receivers:
  otlp:
    protocols:
      grpc:
      http:
exporters:
  jaeger:
    endpoint: "jaeger-collector:14268"
    # en local podemos usar el jaeger instalado en el cluster o exportar al
    local
```

(Alternativa: si no quieres collector, la app puede exportar directamente a Jaeger; el Collector es la opción más profesional.)

7) Desplegar la app de pagos (Docker + Kubernetes)

Opciones para la imagen:

1. **Construir dentro de minikube** (recomendado, no necesitas Docker Hub):

Shell

```
# desde la raíz /app
minikube image build -t payments-demo:latest -f app/Dockerfile .
```

ó

Shell

```
eval $(minikube -p minikube docker-env)
docker build -t payments-demo:latest -f app/Dockerfile .
```

2.

(Alternativa) Push a Docker Hub y usar imagen `<usuario>/payments-demo:tag`.

Manifests Kubernetes en `/app/k8s/` (deployment + service + servicemonitor). Ejemplo `deployment.yaml`:

```
None
apiVersion: apps/v1
kind: Deployment
metadata:
  name: payments
  labels:
    app: payments
spec:
  replicas: 2
  selector:
    matchLabels:
      app: payments
  template:
    metadata:
      labels: { app: payments }
    spec:
      containers:
        - name: payments
          image: payments-demo:latest
          ports:
            - containerPort: 8080
              name: http
          env:
            - name: OTEL_EXPORTER_OTLP_ENDPOINT
              value: "http://otel-collector:4317"
            - name: OTEL_SERVICE_NAME
              value: "payments"
```

`service.yaml` (exponer puerto y nombrar el puerto para ServiceMonitor):

```
None
apiVersion: v1
kind: Service
metadata:
  name: payments
  labels:
    app: payments
```

```
spec:
  selector:
    app: payments
  ports:
  - protocol: TCP
    port: 8080
    targetPort: 8080
    name: metrics
```

`servicemonitor.yaml` para que Prometheus scrapee `/metrics`:

```
None
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: payments-servicemonitor
  labels:
    release: kube-prometheus-stack
spec:
  selector:
    matchLabels:
      app: payments
  namespaceSelector:
    matchNames: [ "payments" ]
  endpoints:
  - port: metrics
    path: /metrics
    interval: 15s
```

Puedes aplicar con `kubectl apply -f app/k8s/` o convertir esos manifests a recursos `kubernetes_*` en Terraform (si quieres todo por Terraform).

8) Instrumentar la app para métricas y traces (cambios mínimos a `app/main.py`)

Tu snippet ya exporta métricas Prometheus (Histogram). Te recomiendo añadir:

- Un Counter `http_requests_total` para alertas por ratio 5xx.
- OpenTelemetry OTLP exporter para traces.

Ejemplo `requirements.txt` (añadir):

```
None
fastapi
uvicorn
prometheus_client
opentelemetry-sdk
opentelemetry-exporter-otlp-proto-grpc
opentelemetry-instrumentation-fastapi
```

Fragmento para `main.py` (añade traces + counter):

```
Python
# imports previos...
from prometheus_client import Counter, Histogram, generate_latest,
CONTENT_TYPE_LATEST
from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.resources import Resource
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import
OTLPSpanExporter
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor

# metrics
REQUEST_TIME = Histogram("http_request_duration_seconds", "Duración ...",
["route", "code", "method"], buckets=(0.01,0.05,0.1,0.25,0.5,1,2,5))
REQUEST_COUNT = Counter("http_requests_total", "Contador de requests",
["route", "code", "method"])

# OTEL setup (env OTEL_EXPORTER_OTLP_ENDPOINT expected, ej:
http://otel-collector:4317)
otlp_endpoint = "otel-collector:4317" # o from env
trace.set_tracer_provider(TracerProvider(resource=Resource.create({"service.name": "payments"})))
otlp_exporter = OTLPSpanExporter(endpoint=otlp_endpoint, insecure=True)
trace.get_tracer_provider().add_span_processor(BatchSpanProcessor(otlp_exporter))
```

```

tracer = trace.get_tracer(__name__)

app = FastAPI()
FastAPIInstrumentor.instrument_app(app,
tracer_provider=trace.get_tracer_provider())

# handlers: increment counter and observe histogram
@app.get("/pay")
def pay(delay: int = 50, fail: int = 2, response: Response = None):
    start = time.time()
    try:
        time.sleep(delay / 1000)
        if random.randint(1,100) <= fail:
            response.status_code = 500
            return {"status": "payment_error"}
        return {"status": "payment_ok"}
    finally:
        duration = time.time() - start
        REQUEST_TIME.labels("/pay", str(response.status_code),
"GET").observe(duration)
        REQUEST_COUNT.labels("/pay", str(response.status_code), "GET").inc()

```

Con eso: métricas + counter + traces OTLP.

9) Alertas (PrometheusRule) – ejemplos (iac/terraform/local/prometheus_rules.yaml)

Ejemplos de alertas importantes (latencia p95, tasa 5xx, caída de pods):

`prometheus-rule.yaml` (PrometheusRule CRD):

```

None
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule

```

```

metadata:
  name: payments-alerts
  labels:
    release: kube-prometheus-stack
spec:
  groups:
  - name: payments.rules
    rules:
      - alert: HighErrorRate
        expr: |
          sum(rate(http_requests_total{job="payments"}[5m])) by (job, code)
          unless on(code) code =~ "5.."
          # (mejor: definimos ratio error sobre total; ajustar expr según métricas
          reales)
        for: 2m
        labels:
          severity: critical
        annotations:
          summary: "High 5xx rate on payments service"

      - alert: HighLatencyP95
        expr: |
          histogram_quantile(0.95,
          sum(rate(http_request_duration_seconds_bucket{job="payments"}[5m])) by (le))
          > 0.5
        for: 2m
        labels:
          severity: warning
        annotations:
          summary: "High p95 latency (>500ms)"

```

(Recomiendo validar las expresiones con las métricas exactas que exporta tu app: counters/histogram names.)

10) SLOs / SLIs / Error budget (archivo: [/docs/slo_sli.md](#))

Ejemplo resumido:

- **SLI:** Request success rate (HTTP code < 500) per 30d; Latency p95.
- **SLO:** 99.9% successful requests over 30 días. (Error budget = 0.1% of requests)
- **SLO (latency):** p95 < 500ms (objetivo).
- **Cómo medir:**
 - $\text{success_rate} = 1 - (\text{sum}(\text{rate}(\text{http_requests_total}\{\text{code}=\sim"5..\"}\{30\text{d}\}))) / \text{sum}(\text{rate}(\text{http_requests_total}\{30\text{d}\})))$
 - $\text{p95} = \text{histogram_quantile}(0.95, \text{sum}(\text{rate}(\text{http_request_duration_seconds_bucket}\{5\text{m}\}))) \text{ by } (\text{le})$
- **Justificación:** con tráfico estimado X RPS y tolerancia de errores Y — aquí argumentos basados en datos sintéticos (explica en informe).

Incluye cálculo de error budget (ej. 30 días -> 30243600 segundos; si hay 1M requests, 0.1% = 1,000 failed allowed).

11) Flujo de trabajo + comandos (rápido, ejecutable)

1. Inicia minikube:

Shell

```
minikube start --driver=docker --memory=6g --cpus=2
```

- 2.

Construye imagen dentro de minikube:

Shell

```
cd app
minikube image build -t payments-demo:latest -f Dockerfile .
```

3.

Desde `iac/terraform/local/`:

Shell

```
terraform init
terraform apply -auto-approve
# esto instalará kube-prometheus-stack y otel collector
```

4.

Aplica manifests de la app (si no lo haces con Terraform):

Shell

```
kubectl apply -n payments -f app/k8s/
```

5.

Verifica pods/svc:

Shell

```
kubectl get all -n payments
kubectl get servicemonitor -n payments
```

6.

Port-forward Grafana / Jaeger:

Shell

```
kubectl port-forward svc/kube-prometheus-stack-grafana 3000:80 -n payments
# abrir http://localhost:3000 (user: admin/admin si no cambiaste)
kubectl port-forward svc/jaeger-query 16686:16686 -n payments # si instalaste
jaeger
```

7.

Ejecuta carga:

Shell

```
# ejemplo con hey (instálalo)
hey -z 2m -c 20 -q 50 "http://$(minikube service --url payments -n
payments)/pay?delay=50&fail=2"
```

8.

Recolecta evidencias:

- `kubectl logs -n payments -l app=payments`
- Exportar dashboard desde Grafana (UI -> Share -> Export -> Save JSON)
- Tomar capturas de dashboards y alert rules.

12) Archivos que agregas al repo (lista mínima)

None

```
/iac/terraform/local/providers.tf
/iac/terraform/local/main.tf          # helm_release kube-prom, otel
collector (y opcional jaeger/loki)
/iac/terraform/local/variables.tf
/iac/terraform/local/outputs.tf
/iac/terraform/local/helm-values/*.yaml
/app/k8s/deployment.yaml
/app/k8s/service.yaml
/app/k8s/servicemonitor.yaml
/app/k8s/prometheus-rule.yaml
/app/requirements.txt (añadidos OTEL entries)
```

13) Buenas prácticas y notas sobre backend Terraform

- No mezcles backend remoto (S3) y `-local` en la misma ejecución. Para local, trabaja en `iac/terraform/local/` y ejecuta `terraform init` allí. Así no necesitas tocar tu `backend.tf` global.
 - Mantén `modules/monitoring/` como módulo reusable: en cloud podrías reutilizar la configuración (helm values) para EKS/GKE, sólo cambian providers.
-

14) Checklist final (para tu README de arranque)

En tu README incluye:

- Requisitos (minikube, docker, terraform, helm, kubectl).
 - Pasos exactos (start minikube, build image, `cd iac/terraform/local && terraform apply`).
 - Cómo acceder a Grafana / Prometheus / Jaeger (puertos, credenciales).
 - Cómo ejecutar las pruebas de carga (`ops/load_test.sh`) y qué evidencias recoger.
 - Dónde están los dashboards (ruta `ops/grafana_dashboards/`) y reglas de alertas (yaml).
 - Señalar trade-offs (local vs cloud): ej. en local no hay LB real, no production secrets, backend terraform local.
-

15) Extras sugeridos (bonus)

- Añadir chaos experiment: `ops/chaos.sh` que escala replicas a 0 y registra la recuperación (para justificar política de SRE).
- Integrar tfsec/checkov (GitHub Action) como pipeline de linting IaC.
- Políticas RBAC mínimas para Prometheus/Grafana (si quieres puntos extra en seguridad).

- Cost analysis: local = \$0; cloud = estimado por recurso (documentarlo).